

MCP, 7월 28일 이전과 이후

프로토콜의 과거, 현재, 그리고 준비할 미래

조훈 Hoon Jo AAIF & CNCF Ambassador

AI & Cloud Native Engineer, MegazoneSoft

오픈소스 테크데이 2026 | 10월 7일(수) | 양재 aT센터 세계로룸



발표자 소개



AAIF Ambassador

에이전틱 AI(agentic AI) 생태계의 오픈소스를 알리는 역할.
MCP Dev Summit Seoul에서 AMA 진행



CNCF Ambassador

클라우드 네이티브 쪽에서는 KubeCon과 Open Source Summit에서 발표



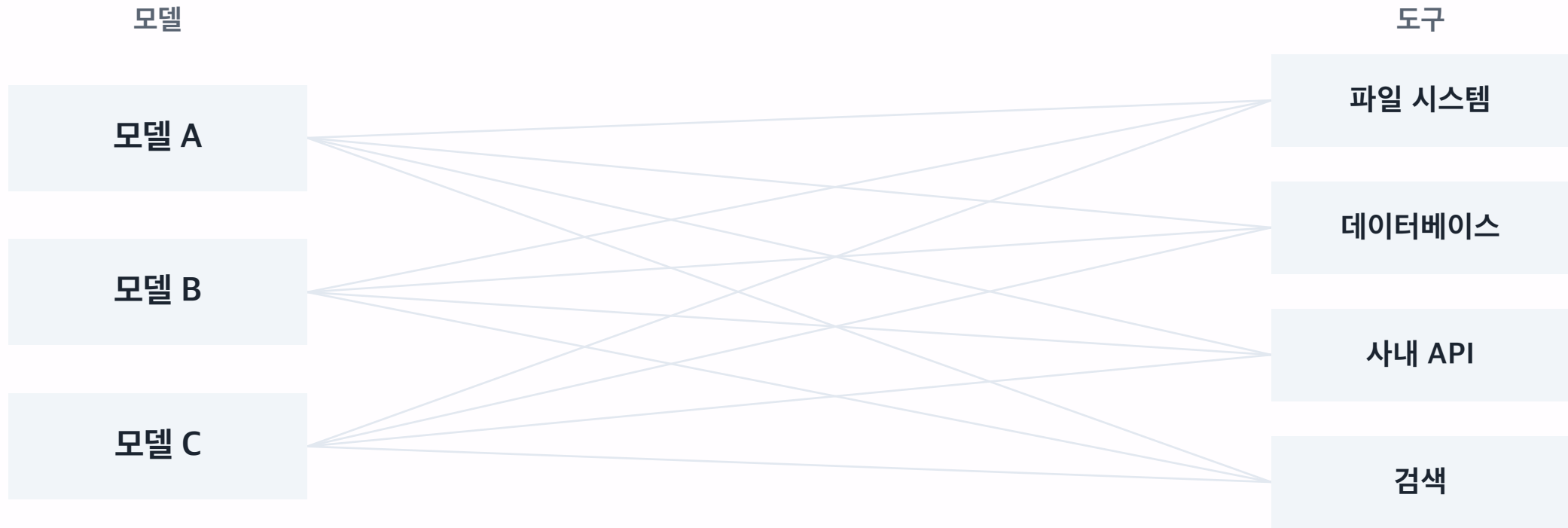
기술서 저자

최신작 두 권은 클로드 코드 책. 그 외 쿠버네티스/도커, 앤서블 등 다수 집필



01

MCP 이전: 모델마다 도구를 따로 연결

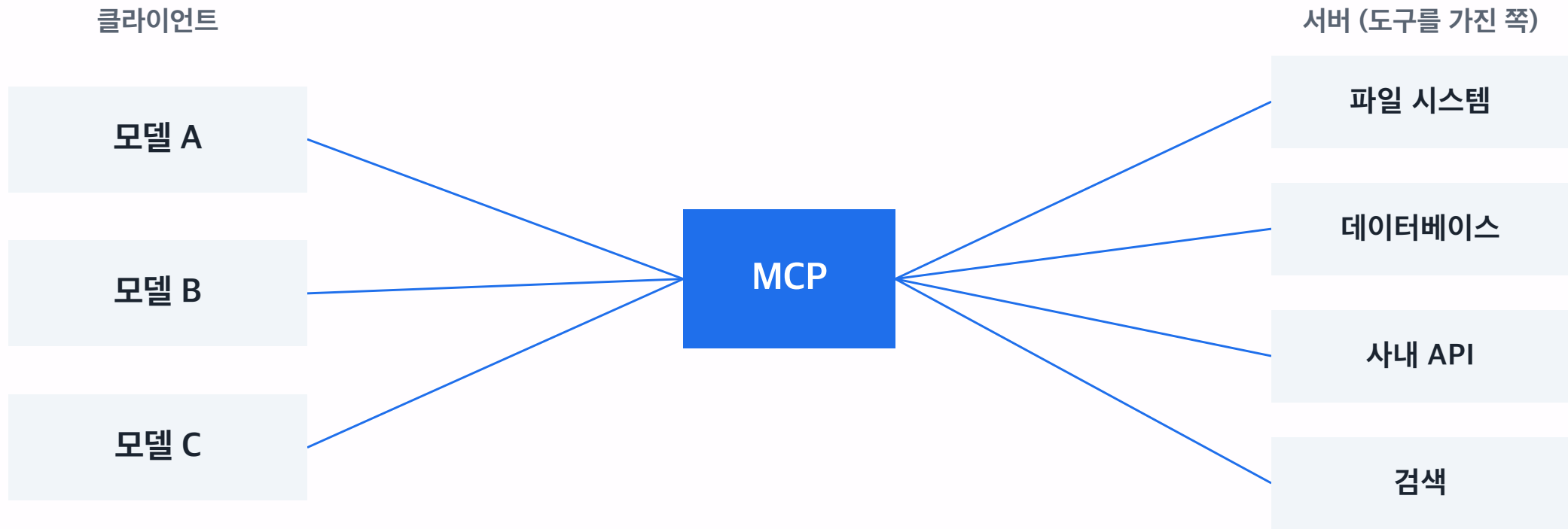


연결마다 어댑터를 따로. **모델 M개 x 도구 N개 = 연결 M x N**

같은 도구라도 모델마다 연결 코드를 다시 만들어야 했습니다.

01

MCP: AI와 도구 사이의 공통 규약



규약을 사이에 하나 두면 **연결이 $M + N$ 으로 줄어듭니다**

서버를 한 번 만들면 어떤 클라이언트든 그 도구를 씁니다.



2024년 공개부터 2026-07-28 개정까지

2024-11

2025-03

2025-11

2025-12

2026-07-28



공개



Streamable HTTP, 인증



구 스펙 확정



AAIF로 이관



가장 큰 개정

그 사이에 주요 모델사들이 채택하고 SDK가 갖춰지고 수천 개의 서버가 생겼습니다.

하지만 세션이 특정 서버에 묶여 있다 보니 여러 대로 늘려 운영하는 순간부터는 문제가 생기기 시작했습니다.



서버를 여러 대로 늘리면 생기던 문제

**요청을 나눠
보낼 수 없음**

세션을 기억하는 서버로만 보내야 함.
스티키 세션이 필요

**서버가 종료되면
세션도 사라짐**

장애가 난 서버가 가지고 있던 세션은
복구되지 않음

**세션 코드는
개발자가 직접 구현**

버그와 메모리 누수가 발생하던 코드

게다가 구현은 제각각이었고 사용 사례의 약 90%는 세션 기능을 쓰지도 않았습니다.

2026-07-28

MCP 공개 이후 가장 큰 개정



스테이트리스: 세션과 핸드셰이크 제거

이전

- initialize 핸드셰이크로 시작
- Mcp-Session-Id로 세션 유지
- 같은 서버가 계속 응답해야 함

2026-07-28 이후

- 요청마다 필요한 정보를 모두 포함
- 핸드셰이크도 세션 ID도 없음
- 어떤 서버가 받아도 그 요청만 보고 처리

웹에서 오래 검증된 요청-응답 모델로 돌아온 것입니다.



통신 방식과 인프라, 인증이 함께 바뀌었습니다

MRTR (다중 왕복 요청)

Multi Round-Trip Requests. 서버가 클라이언트를 직접 호출하는 대신 중간 결과를 돌려주고 다시 요청받음

헤더 라우팅

본문(JSON)을 검사하지 않고 메서드와 도구 이름을 헤더에서 확인

tools/list 응답 캐시

연결이 달라도 같은 목록. 캐시 유효 시간 힌트까지 표준화

OAuth 2.0과 OIDC에 맞춤

기업이 이미 쓰는 인증 방식을 그대로 사용. 발급자(issuer) 검증 추가



새로운 버전을 사용하려면?

하위 호환을 고려했지만 버전 2개를 함께 유지하기 어렵다고 보아 새로운 형태의 버전만 관리하기로 했습니다.
그래서 새로운 버전으로 이전이 필요합니다. (SEP-2575)

핸드셰이크 ➡ discover

- initialize (한 번 맺고 세션 시작)
- + **server/discover** (서버는 필수 구현)

세션 상태 ➡ 핸들

- Mcp-Session-Id: sess-42 (서버가 기억함)
- + **get_cart(cart_id="cart-42")** (인자로 받음)

서버 발신 요청 ➡ MRTR

- elicitation/create (서버가 먼저 보냄)
- + **resultType: "input_required"** (한 번 더 왕복)

리소스 구독 ➡ 한 스트림

- resources/subscribe {uri}
- + **subscriptions/listen {resourceSubscriptions}**

코드는 바뀐 SDK 에 맞춰 고칩니다. 위 4가지는 코어에서 바뀐 대표적인 예시입니다.

오래 걸리는 도구가 있다면 tasks 확장을 따로 선언해야 합니다. 세션이 없으니 인증과 인가는 요청마다 걸어야 합니다.



새로운 버전으로 안 바꾼다면?

당장 문제가 발생하진 않습니다.

강제 전환 시점이 없고 구 스펙 서버는 그대로 동작합니다.

신 스펙 전용 클라이언트가 못 붙습니다.

양쪽을 지원하는 클라이언트는 그대로 연결됩니다.

세션 운영 부담을 그대로 가지고 가야 합니다.

스티키 세션이 필요하고 서버가 종료되면 세션도 사라집니다.

새로운 기능 확장을 사용할 수 없습니다.

tasks, apps, EMA 확장은 신 스펙에만 있습니다.



이 전환이 운영에 주는 의미

원격 MCP 서버를 평범한 웹 서비스처럼 운영할 수 있게 됐습니다.

일반 로드밸런서

스티키 세션 없이 라운드로빈 분배

응답 캐시

결과를 ttlMs 규칙대로 저장

표준 인증

기업이 쓰던 OAuth·OIDC 관행 적용

실험적 프로토콜에서 운영 가능한 인프라로 넘어가는 단계입니다.



두 달이 지난 지금

서버와 플랫폼

깃허브 7월 전환 완료. 클라우드플레어는 세션 인프라 제거

공식 SDK

파이썬은 mcp 2.2, 타입스크립트는 새 패키지 2.0.
기존 1.x도 유지보수

클라이언트

신 스펙 순차 적용 중. 버전별 지원 표는 아직 없음

호환 규칙

신 스펙 전용과 구 스펙 전용이 연결될 때만 실패.
한쪽이 양쪽을 지원하면 동작

강제 전환은 없습니다. 더 이상 사용을 권고하지 않는(deprecated) Roots, Sampling, Logging, DCR 은 2027년 7월까지 동작합니다

예외는 HTTP+SSE 전송입니다. 2025년 3월부터 예고돼 이미 유예 기간이 지났습니다.

직접 돌려 보죠

LIVE DEMO 도구 2개짜리 MCP 서버

왼쪽 화면

두 스펙에 같은 호출을 1초마다 계속 보냅니다

watch_calls.sh

오른쪽 화면

단계마다 명령을 하나씩 실행합니다

1.demo.sh

서버 4대가 이미 실행 중입니다. 8100과 8101은 구 스펙, 8200과 8201은 신 스펙입니다.

LIVE DEMO 같은 것을 두 번 보고 무엇이 달라지는지 봅니다

1

서버 코드를 봅니다

4대가 같은 파일이고 플래그 하나만 다릅니다

2

세션 헤더를 넣고 빼며 호출합니다

같은 호출 4번 중 구 스펙만 한 번 막힙니다

3

서버를 죽입니다

예비가 남았는데 구 스펙만 끊깁니다

4

MCP를 연결합니다

모르던 것을 답하고 값까지 바꿉니다

여기서부터는 터미널에서 실제로 실행해 보겠습니다.

데모 코드

sysnet4admin/talks
ETRI-OST/2026-TechDay





방금 보신 것

서버는 수십 줄

규약이 정해져 있어 도구만 작성

데모의 `ost_server.py`: 76줄

연결은 설정 몇 줄

클라이언트가 규약대로 도구를 발견

데모의 `mcp.json`: 8줄

읽기도 쓰기도 에이전트

쓸 도구를 사람이 정하지 않음

`get_project`, `update_project`



앞으로의 방향

코어는 더 작아집니다

더 이상 사용을 권고하지 않는
기능부터 빠집니다

다음 차례는 HTTP+SSE 전송

확장이 그 자리를 채웁니다

워킹그룹이 코어 밖에서
표준을 만듭니다

워킹그룹 12개, 최근 2개 추가

운영은 더 평범해집니다

일반 웹 트래픽처럼 관리합니다

OpenTelemetry 추적 포함

예측은 여기까지만 하겠습니다. 스펙은 앞으로도 계속 개정됩니다.



04 지금 시작한다면

새 서버는 신 스펙으로 시작

구 스펙으로 새로 만들 이유가 없음. 스테이트리스 전제로 설계

기존 서버는 2027년 7월 유예 안에 계획 수립

지금은 전부 동작. 파이썬과 타입스크립트는 양쪽 지원이 기본값

공식 SDK부터 확인

파이썬 mcp 2.x, 타입스크립트 새 패키지. 문서는 modelcontextprotocol.io

바로 시작

파이썬

```
pip install mcp
```

타입스크립트

```
npm i @modelcontext  
protocol/server
```

연구실에서 쓰는 도구 하나를 서버로 만들어 보는 것이 시작하기 가장 좋은 방법 중 하나입니다.



04 코어 밖의 프로젝트와 표준

같은 재단에서 제공하는 다른 종류의 프로젝트

MCP	에이전트와 도구 사이
A2A	에이전트와 에이전트 사이
agentgateway	MCP와 A2A 를 기본 지원
Agent Router	모델 호출을 한 API 로 통일

AAIF 프로젝트 6개 (aaif.io/projects)

관측은 OpenTelemetry로 진행

_meta 전파	traceparent, tracestate, baggage
MCP 규약	mcp.method.name, mcp.protocol.version
상태	Development, 아직 안정 버전 아님

SEP-414 (2026-07-28) / semconv-genai

MCP 는 도구 연결까지 담당합니다.

그 외에는 다른 프로젝트들과 OpenTelemetry와 같은 다른 재단(CNCF)의 표준이 진행합니다.



세 문장으로

과거

도구 연결을 규약으로 정리

모델마다 도구를 따로 연결하던 것이 M + N으로

2024-11 공개 ~ 2026-07-28

현재

요청-응답 모델로 전환 중

실험적 프로토콜에서 운영 가능한 인프라로 이동

지금, 생태계 전환 진행 중

미래

코어는 축소, 확장은 별도

선택 기능은 확장이 담당하고 필요한 것만 선택

2026-08 로드맵, 2027년 이후

감사합니다

스펙 문서와 발표자 정보는 아래 QR로 확인하실 수 있습니다.



스펙과 문서
modelcontextprotocol.io



발표자
linkedin.com/in/hoonjo

조훈 Hoon Jo AAIF & CNCF Ambassador | 오픈소스 테크데이 2026

